TYPING SPEED TEST – PROJECT

PROJECT TITLE : TYPING SPEED TEST

COURSE TITLE : FUNDAMENTALS IN AI AND ML

COURSE CODE : CSA2001

COURSE TYPE : FLIPPED COURSE

COURSE CREDITS : 4

PROFESSOR : DR SUBASH CHANDRA BOSE J

SLOT : A21+A22+A23

NAME : S G SUDHARSHAN

REGISTRATION NO. : 25BAI10706

SUBMISSION DATE : 30/3/2026

INTRODUCTION

Project Overview

In the modern digital age, typing has evolved from being a highly specialized skill for office work to a basic personal skill for everyone. Whether for academic writing, professional communication, data entry, programming, or even casual web surfing, being able to type is now a highly valuable skill. Typing Speed Test is a software tool that is intended to test the efficiency of a user’s typing skills based on two key parameters: speed, expressed in terms of Words Per Minute (WPM), and accuracy, expressed as a percentage.

Although there are many typing tests on the web today that can be used to determine one's typing speed, they are mostly heavy and depend on web trackers and complex Graphical User Interfaces (GUIs). This project provides a lightweight Typing Speed Test written in Python that runs in the terminal. The aim is to eliminate all the visual fluff and get to the bare computational essence of the task.

The program selects random sentences from an array list provided in advance, asks the user to input them, and uses the internal clock of the computer to measure real-time performance. The program is an interface between basic programming variables and practical utility programs.

PROBLEM STATEMENT

The majority of the computer science students of the current generation are comfortable with the pre-built digital tools and software packages available for use. However, the gap that seems to exist is the tendency of the students to become passive users of the technology, as opposed to actively creating it, while at the same time using the software packages on a daily basis without the knowledge of the mathematical and logical operations that are going on behind the scenes.

The problem that the project seeks to solve can be identified as follows:

1.To create a localized utility that does not depend on the use of the internet bandwidth.

2.To understand the system architecture of the timestamping and string parsing to calculate the human performance metrics programmatically.

PROJECT OBJECTIVES

The main goals for the development of this Python Terminal Typing Speed Test program are as follows:

- To Develop a Functional CLI Utility: To develop a Command Line Interface utility using Python's native time and random modules without having to depend heavily on external and heavy-weight third-party modules.
- To Implement Accurate Real-Time Time-stamping: To make accurate epoch time calculations to ensure that the calculations are fair and standardized.
- To Evaluate Algorithmic Accuracy checks: To develop a comparative logical loop that can compare user input string literals to standard prompt string literals.
- To Foster Foundational Python Logic: To leverage basic programming concepts and techniques such as basic I/O operations, basic data structures, basic logical control flow statements, and basic arithmetic operators to solve a real-world problem.

FUNCTIONAL REQUIRMENTS

- Prompt Generation: The system must be able to generate the target sentence randomly and display it.
- Input Mechanism: The system must wait and pause until the user enters the input by pressing "Enter."
- Text Capture: The system must capture the input text of the user in real-time.
- Time Tracking: The system must track the time accurately to calculate the time taken by the user.
- Metric Calculations: The system must calculate the metrics using the data:
- Words Per Minute (WPM): The system must calculate the time taken by the user using the division of the total number of characters by the time taken.
- Accuracy Percentage: The system must calculate the accuracy using the exact matching of the text.
- Result Display: The system must display the time taken in seconds, speed in WPM, and accuracy in percentage on the console.

NON FUNCTIONAL REQUIRMENTS

Performance and Response Time

- Instantaneous Processing: The system needs to process information instantaneously after the user hits the "Enter" button by processing information and computing values within less than 1 second.

- Low Resource Usage: The application needs to utilize fewer CPU resources and less than 50MB of RAM in order to function smoothly without lag in the background.

Usability and Human Interface

- Clear Readability: The terminal needs to display information in such a way that it does not cause eye strain to the user while reading the target prompt displayed.

- Intuitive Operation: The application needs to have a zero-learning curve so that even first-time users can run the application/script successfully without needing further manuals.

Reliability and Error Forgiveness

- Deterministic Computations: If two different users input the same information in terms of time taken and characters typed, then the WPM and accuracy values must always remain mathematically equivalent.

- Edge-Case Stability: If the user hits the "Enter" button without typing in any text by mistake, then the application needs to function properly without crashing.

Portability and Maintainability

- Cross-Platform Compatibility: Using only native Python modules, it needs to function flawlessly in all Windows, macOS, and Linux terminal platforms.

IMPLEMENTATION DETAILS

The execution of the Typing Speed Test is done through the use of Python 3.x. The application is totally based on native and built-in libraries that come with Python. This means that there is no need to install any third-party libraries (using pip install). This has made the application very lightweight and fast.

Here are some architectural and programming details on how this software was written.

Libraries and Modules Used

The application imports two basic modules that come with Python.

- time Module: This is an important module for capturing system timestamps. The time.time() function is used to capture the epoch time with very high accuracy because it can capture fractional seconds in floating-point numbers.
- random Module: This module is used to create dynamic elements in the application. The random.choice() function is used to select a random sentence from a given list without any human intervention.

SCREENSHOTS & RESULTS

```python
import time
import random

print("Typing Speed Test")
print("------------------")

# some sentences
lines = [
    "python is easy to learn",
    "practice makes a person perfect",
    "coding is fun when you understand it",
    "i am learning python",
    "speed comes with practice"
]

# pick one line randomly
line = random.choice(lines)

print("\nType this line:")
print(line)

input("\nPress Enter when you are ready...")

# start time
start = time.time()
```

```python
# user types
user_text = input("\nStart typing:\n")

# end time
end = time.time()

# total time
time_taken = end - start

# word count
words = len(user_text.split())

# speed calculation
wpm = (words / time_taken) * 60

# accuracy check
correct = 0
for i in range(len(line)):
    if i < len(user_text) and line[i] == user_text[i]:
        correct += 1

accuracy = (correct / len(line)) * 100

# result
print("\nResult")
print("Time taken:", round(time_taken, 2), "seconds")
```

```python
# accuracy check
correct = 0
for i in range(len(line)):
    if i < len(user_text) and line[i] == user_text[i]:
        correct += 1

accuracy = (correct / len(line)) * 100

# result
print("\nResult")
print("Time taken:", round(time_taken, 2), "seconds")
print("Speed:", round(wpm, 2), "words per minute")
print("Accuracy:", round(accuracy, 2), "%")
```

RESULT:

```
Typing Speed Test
*------------------*

Type this line:
python is easy to learn

Press Enter when you are ready!!!

Start typing:
 python is easy to learn

Result
Time taken: 14.15 seconds
Speed: 21.2 words per minute
Accuracy: 100.0 %
```

REFERENCES

1. VIT BHOPAL Academic guidelines for grading and credit calculation.
2. VITYARTHI and CSA2001 course
3. Google for learning about Documentation.
4. Peer and mentor feedbacks from,
   - DR SUBASH CHANDRA BOSE J
   - GOKUL PRASAD K– 25BAI10148
   - DILIP S – 25BAI10590
   - BAVESH P –25BAI10093
   - RAGHUL SK – 25BAI10288

THANK YOU